TP Argumentation Theory

Dapzol Nicolas / Jebari Faik / Nguyen Tony-Cuong

Part 1: Reasoning with ABA.....2
1. Méthode générale..... 2
2. Framework ABA développé.....2
3. Fonctions développées..... 3
3.1. Recherche des arguments..... 3
3.2. Recherche des attaques :..... 3
3.3. Génération d'arguments atomiques..... 4
3.4. Transformation en non circular.....4
4. Résultats..... 5
4.1. Résultats sur la génération arguments et attaques.....5
4.2. Résultats sur la transformation de circular à non circular et génération atomic framework.....6
5. Appli en ligne.....7
5.1. Méthode et lien.....7
5.2. Utilisation.....7
5.3. Tests.....7
Part 2 : Relation-based argument mining..... 8
Question 1: Méthode de scraping..... 8
Question 2 : Classification et accuracy..... 8
Part 3: Visualising acceptability degree spaces for gradual semantics..... 11
Question 1: Weighted h-categorizer with a convergence parameter ε..... 11
Question 2: Acceptability degree spaces..... 12
(a, b) Frameworks and sampling.....12
(c) Convex hull..... 12
(d) Heat map of the density..... 13
Conclusion.....15

Introduction

Ce rapport s'intéresse à la théorie de l'argumentation sous trois aspects : 1) l'implémentation d'un framework ABA, 2) la recherche et la typologie d'arguments dans l'analyse d'une base réelle 3) A partir d'arguments, la définition des différents degrés d'acceptabilité et leurs visualisations.

L'ensemble du code associé est disponible dans le fichier zip associé (ou dans le github ici https://github.com/nicolasdapzol/IA_AT_TP)
La participation de chacun est enregistrée dans le fichier "involvement".
L'application de la partie 1) est disponible via https://ia-at-tp.vercel.app/

Part 1: Reasoning with ABA

Cette partie vise à reproduire implémenter le framework ABA en Python

1. Méthode générale

Pour l'algorithme nous nous sommes inspirés largement du CM et des TD que nous avons déployé en Python.
Pour la structure du code, nous avons fait plusieurs essais. Les premiers frameworks développés ne permettaient pas simplement d'aller jusqu'au transformation circulaire-> non circulaire ou ne gérait pas les attaques inversées.
Nous avons ensuite utilisé Copilot pour avoir des idées sur la façon d'organiser le code en python et les différentes classes à construire (notamment l'utilisation de la bibliothèque dataclasses avec l'option frozen=true) et en cas de blocage important (notamment la partie non_circular to circular). Néanmoins, pour toutes les parties produites par IA, le code a été revu, simplifié et annoté.
La majeure partie du code est manuel et est disponible ici :

https://github.com/nicolasdapzol/IA_AT_TP

Pour l'appli en ligne, pour la génération d'un code cohérent avec le site, nous avons fait appel à Copilot aussi. Nous avons instancié le modèle en local, élaboré le cahier des charges et les spécifications, demandé à copilot de le rendre compatible avec le site, corrigé et vérifié les résultats sur 5 cas.

https://ia-at-tp.vercel.app/

Nous sommes conscients que pour une mise en production réelle, les tests effectués seraient insuffisants.

Par la suite, nous proposons une implémentation en python

- du framework ABA.
- des outils pour générer des arguments et des attaques
- d'outils pour convertir un framework en un framework non circulaire et atomic
- les résultats
- l'appli en ligne

2. Framework ABA développé

L'objet ABA_generator comporte la définition des éléments du framework :

- literals
- rules

```
aba = ABA_generator(
    literals=['a', 'p','q'],
    assumptions=['a'],
    contraries={'p': 'a'},
    rules=[('r1', 'q', ['p']),
           ('r2', 'p', ['a'])
           ],
    preferences=[]
)
```

- assumptions,
- contrary
- les préférences

Pour faciliter la construction, les règles sont des objets à part entière ainsi que les arguments et les attaques.

L'ensemble est défini dès la création de l'objet et sont immuables.
3 imports sont possibles (directement, par fichier ou par zone de textes).
A noter, il n'y pas de vérification à l'entrée des variables mobilisées : cela serait une amélioration souhaitable dans le futur.

3. Fonctions développées

3.1. Recherche des arguments

on initialise avec les assumptions et les règles avec un corps vide

```python
# assumptions
arguments = {Argument(assumption, frozenset({assumption})) for assumption in assumptions}
# faits
arguments.update(
    Argument(rule.head, frozenset()) for rule in selected_rules if not rule.body
)
```

Puis on génère des nouveaux arguments, en itérant sur les règles.
Pour chaque règle, on regarde si on peut générer un nouvel argument à partir des arguments préalablement générés et on itère jusqu'à ce qu'il n'y ai plus de changement.

```python
# tant qu'on peut trouver des nouveaux arguments , on itère
changed = True
while changed:
    changed = False
    for rule in selected_rules:
        # pour chaque règle, on cherche à combiner les supports des littéraux du corps
        support_options = []
        candidat=False
        for literal in rule.body:
            candidat_literal=False
            for argument in arguments:
                if argument.conclusion == literal:
                    support_options.append(argument.support)
                    candidat_literal=True
            if not candidat_literal:
                candidat=False
        # si aucune combinaison n'est possible, on passe à la règle suivante
        if candidat==False:
            continue
        # sinon, on génère toutes les combinaisons possibles des supports des littéraux du corps
        # chaque combinaison de supports choisis correspond à un nouvel argument potentiel
```

3.2. Recherche des attaques :

On opte ici pour une double boucle : pour tous les attaquants parmis les arguments et tous les cibles :

- Si au moins un support de la cible est le contrary de l'attaquer, alors il y a attaque.
- Si la cible est préférée à l'attaquant , l'attaque est inversé.

```python
for attacker in argument_list:
    # pour chaque argument  on le considère comme cible, on vérifie s'il est attaqué par l'argument attaquant
    for target in argument_list:
        # est ce que la conclusion de l'attaquant est le contraire d'une hypothèse du support de la cible
        for support_cible in target.support:
            contrary = framework.contraries.get(support_cible)
            if contrary != attacker.conclusion:
                continue
            # y a une attaque !
            # est elle inversée ?
            # Inversion si l'hypothèse visée est préférée à un élément du support attaquant.
            # l'attaque est inversée
            is_reversed = any(
                (support_cible, support_attaquant) in preferred_over
                for support_attaquant in attacker.support
            )
            if is_reversed:
                attacks.add(Attack(target, attacker, "inversée"))
            else:
                attacks.add(Attack(attacker, target, "normale"))
return attacks
```

3.3. Génération d'arguments atomiques

Pour cela, on crée les arguments avec la fonction précédente.
puis on transforme les arguments en règle.

Enfin, on génère un ABAframework a partir de ceci.

```
rules = [
    Rule(
    for i                                                    e, start=1)
]
```

```
atomic_aba = ABAFramework(
        literals=set(framework.literals),
        assumptions=set(framework.assumptions),
        contraries=dict(framework.contraries),
        rules=list(rules),
        preferences=set(framework.preferences),
    )
```

3.4. Transformation en non circular

Pour convertir un un framework circulaire en non circulaire, 3 étapes sont nécessaires :

1. la création des nouveaux littéraux

on les enregistre ici dans un dictionnaire pour y avoir accès par la suite.

```
nouveau_literals = set(literals)
copies: dict[tuple[str, int], str] = {}
for literal in sorted(literals - assumptions):
    for index in range(1, k + 1):
        copied_literal = f"{literal}^{index}"
        nouveau_literals.add(copied_literal)
        copies[literal, index] = copied_literal
```

2. la dérivation des règles pour les règles préalablement atomiques

On suit la logique du CM 4A.

```
# du CM 4A
# - for each **atomic** rule $r = s \leftarrow p_1, \dots, p_n$:
# - We create $r_1, r_2, \dots, r_k$, where $r_i = s^i \leftarrow p_1, \dots, p_n$ and $s^k  =s$

for rule in framework.rules:
    atomic = _rule_is_atomic(rule, assumptions)

    if atomic:
        # pour chaque règle atomique, on crée k copies avec des les meme body / les noms des règles changent ainsi que les tetes des règles
        for index in range(1, k + 1):
            if index==k:
                head = rule.head
            else:
                head = copies[rule.head, index]
            body = rule.body
            name = f"{rule.name}_{index}"
            converted_rules.append(Rule(name, head, body))
```

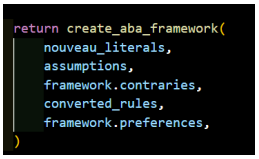
3. la dérivation des règles pour les règles préalablement non atomiques

```
# DU CM :
# - for each **non-atomic** rule r = s <- p_1, , p_n:
# - we create r_2, , r_k, where r_i = s^i <- p'_1, , p'_n and s^k = s and:
#     - $p'_j = p_j$ if $p_j \in A$
#     - $p'_j = p_j^{i-1}$ otherwise.
    else:
        # pour chaque règle non-atomique, on crée k-1 copies avec des bodies modifiés selon les hypothèses et les copies précédentes

        for index in range(2, k + 1):
            if index == k:
                head = rule.head
            else:
                head = copies[rule.head, index]

            premises_nouvelles = []
            for premise in rule.body:
                if premise in assumptions:
                    # si le prémisse est une hypothèse, on le garde tel quel
                    premises_nouvelles.append(premise)
                else:
                    premises_nouvelles.append(copies[premise, index - 1])
            body = tuple(premises_nouvelles)
            name = f"{rule.name}_{index}"
```

On retourne le framework créé.



4. Résultats

Nous avons testé les algorithmes sur 5 exemples issus du CM et des TDs.

- 3 exemples nous servent à tester la génération d'arguments et d'attaques.
- 2 exemples nous servent à tester la transformation du framework en non circulaire et non atomique.

Les résultats sont enregistrés dans le fichier jupyter "rapport_exemples_aba".ipynb
La synthèse des résultats est ci-dessous.

4.1. Résultats sur la génération arguments et attaques

description framework	arguments générés	attaques générés	conforme
## Literals L: [a,b,c,p,q,r,s,t] ## Assumptions A: [a,b,c] ## Contraries C(a): p C(b): s C(c): t ## Rules [r1]: p <- q,a [r2]: q <- [r3]: r <- b,c	1. a (support : {a}) 2. b (support : {b}) 3. c (support : {c}) 4. p (support : {a}) 5. q (support : vide) 6. r (support : {b, c})	1. p (support : {a}) --[normale]--> a (support : {a}) 2. p (support : {a}) --[normale]--> p (support : {a})	oui
## Literals L: [a,b,q,p,r] ## Assumptions A: [a,b] ## Contraries C(a): r C(b): p ## Rules [r1]: p <- q [r2]: q <- a [r3]: r <- b	1. a (support : {a}) 2. b (support : {b}) 3. p (support : {a}) 4. q (support : {a}) 5. r (support : {b})	1. p (support : {a}) --[normale]--> b (support : {b}) 2. p (support : {a}) --[normale]--> r (support : {b})	oui
## Literals L: [a,b,c,q,p,r,s,t] ## Assumptions A: [a,b,c] ## Contraries C(a): r C(b): s C(c): t ## Rules [r1]: p <- q,a [r2]: q <- [r3]: r <- b,c	1. a (support : {a}) 2. b (support : {b}) 3. c (support : {c}) 4. p (support : {a}) 5. q (support : vide) 6. r (support : {b, c}) 7. s (support : {a, c}) 8. t (support : {a, c})	1. a (support : {a}) --[inversée]--> r (support : {b, c}) 2. p (support : {a}) --[inversée]--> r (support : {b, c}) 3. s (support : {a, c}) --[normale]--> b (support : {b}) 4. s (support : {a, c}) --[inversée]--> r (support : {b, c}) 5. s (support : {a, c}) --[normale]--> r (support : {b, c}) 6. t (support : {a, c}) --[normale]--> c (support : {c}) 7. t (support : {a, c}) --[inversée]--> r (support : {b, c}) 8. t (support : {a, c}) --[normale]--> r (support : {b, c}) 9. t (support : {a, c}) --[normale]--> s (support : {a, c}) 10. t (support : {a, c}) --[normale]--> t (support : {a, c})	oui

[r4]: t <- p,c [r5]: s <- t ## Preferences PREF: a > b			

4.2. Résultats sur la transformation de circular à non circular et génération atomic framework

description framework	non circular	atomic framework	conformes
## Literals L: [a,p,q] ## Assumptions A: [a] ## Contraries ## Rules [r1]: p <- a [r2]: p <- q [r3]: q <- p ## Preferences PREF: a > b	L: [a, p, p^1, p^2, q, q^1, q^2] A: [a] Contraires : (aucun) Règles : [r1_1]: p^1 <- a [r1_2]: p <- a [r2_2]: p <- q^1 [r3_2]: q <- p^1 Préférences : a > b	L: [a, p, p^1, p^2, q, q^1, q^2] A: [a] Contraires : (aucun) Règles : [r_1]: a <- a [r_2]: p <- a [r_3]: p^1 <- a [r_4]: q <- a Préférences : a > b	oui
## Literals L: [a,b,x,y,z] ## Assumptions A: [a,b] ## Contraries C(a): y C(b): z ## Rules [r1]: y <- b [r2]: y <- y [r3]: x <- x [r4]: x <- a [r5]: z <- x,y ## Preferences PREF: a > b	L: [a, b, x, x^1, x^2, x^3, y, y^1, y^2, y^3, z, z^1, z^2, z^3] A: [a, b] Contraires : C(a): y C(b): z Règles : [r1_1]: y^1 <- b [r1_2]: y^2 <- b [r1_3]: y <- b [r2_2]: y^2 <- y^1 [r2_3]: y <- y^2 [r3_2]: x^2 <- x^1 [r3_3]: x <- x^2 [r4_1]: x^1 <- a [r4_2]: x^2 <- a [r4_3]: x <- a [r5_2]: z^2 <- x^1, y^1 [r5_3]: z <- x^2, y^2	L: [a, b, x, x^1, x^2, x^3, y, y^1, y^2, y^3, z, z^1, z^2, z^3] A: [a, b] Contraires : C(a): y C(b): z Règles : [r_1]: a <- a [r_2]: b <- b [r_3]: x <- a [r_4]: x^1 <- a [r_5]: x^2 <- a [r_6]: y <- b [r_7]: y^1 <- b [r_8]: y^2 <- b [r_9]: z <- a, b [r_10]: z^2 <- a, b Préférences : a > b	oui

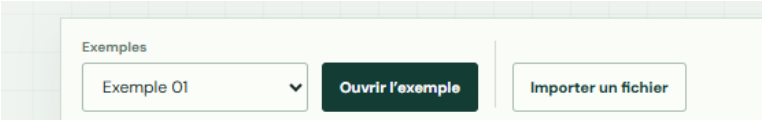
5. Appli en ligne

5.1. Méthode et lien

L'application a été mise en ligne sur Versel et synchronisée avec GitHub.

le lien fonctionnel est ici : https://ia-at-tp.vercel.app/

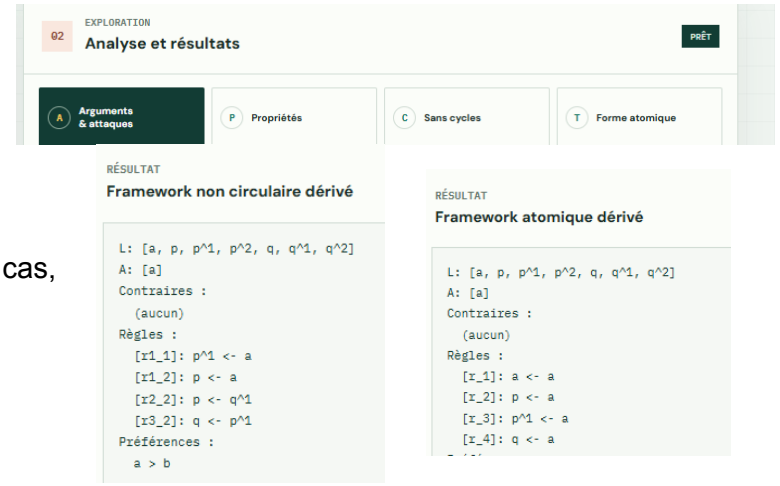
5.2. Utilisation



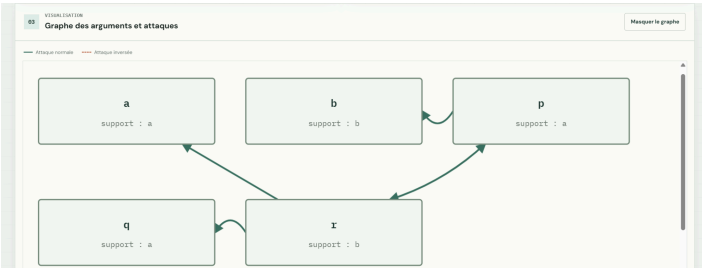
L'importation du framework se fait soit en ouvrant une exemple préétabli soit en important un fichier. le framework est modifiable directement dans l'éditeur de texte si besoin.

Différente actions sont possibles :

- Calculer les arguments et les attaques
- Vérifier si le graph est circulaire ou atomique
- Transformer en non circulaire
- Transformer en atomique(ds ce cas, le grah est automatiquement transformé en non circulaire)



L'affichage du graph des arguments et des attaques est possible.



5.3. Tests

Les 5 exemples fonctionnent correctement. D'autres tests de validités d'entrée pourraient être mis en œuvre.

Part 2 : Relation-based argument mining

Question 1: Méthode de scraping

Le dataset que l'on cherche se trouve sur le site Kialo et est obtenu à l'aide d'une requête **GET** sur chacun des discours afin de récupérer une dataframe à l'aide de la bibliothèque panda permettant de les ordonner dans un fichier **csv**. Le dossier ***kialo_export*** va alors contenir à la fois le dossier ***debates*** qui contient lui même chacun des fichiers .txt après tous les get, ainsi que le csv ***kialo-top250-discussions.csv*** qui contient les tables des 250 débats qui vont être télécharger dans le dossier ***debates.***

```
Successfully fetched and processed 250 discussions.
       id                                              title  \
0  29355                       Are Purity Pledges Harmful?
1  10143              Should American Football Be Banned?
2  30339                 Should there be One World State?
3  31027  Do fair trade products cause more good than harm?
4  64072  Should failing banks receive bailouts from the...

                                       kialoUrlId  \
0                 are-purity-pledges-harmful-29355
1          should-american-football-be-banned-10143
2             should-there-be-one-world-state-30339
3  do-fair-trade-products-cause-more-good-than-ha...
4  should-failing-banks-receive-bailouts-from-the...

                                             tags
0           Purity, Sex, Virginity, Feminism, Women
1  Sports, USA, Football, American_Football, Ente...
2     States, Nations, Culture, Economics, Politics
3         Trade, Development, International, Ethics
4  Economy, Finance, States, Government, Banks, B...
```

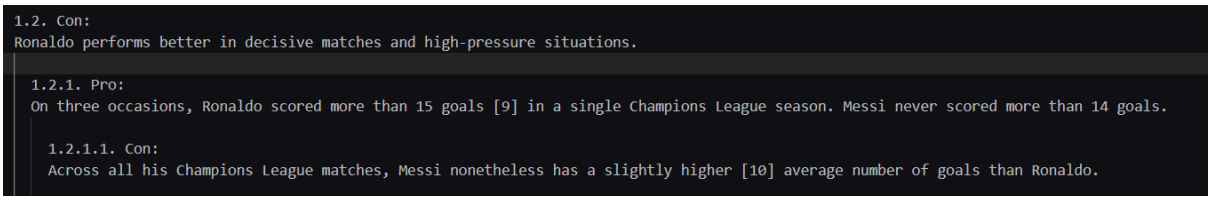
Une fois le fichier **csv** obtenu on récupère chacun des arguments individuellement à l'aide de la bibliothèque selenium nous permettant grâce à un compte Kialo de pouvoir télécharger chacun des débats grâce au slug de l'url du débat qui est modifié à chaque itération dans la boucle puis d'être mis dans un dossier chacun au format **.txt** pour pouvoir être utilisé pour la **classification** ensuite.

Question 2 : Classification et accuracy

Cette partie permet d'analyser le pourcentage d'**accuracy** entre les 250 débats les plus actifs de la plateforme Kialo afin de savoir comment les idées sont argumentés entre **PRO** et **CON** (soutient et réfutation d'une idée).

Tout d'abord les arguments vont être lues dans chacun des débats puis le parser définir les arguments, ensuite la fonction predict va permettre de savoir si deux arguments ont un lien logique entre eux en comparant les paires choisies dans l'ordre et des pairs prises au hasard au sein du même débat afin que l'algorithme puisse mémoriser ceux qui vont ensemble ou non.

exemple concret de paire :



On obtient une paire liant “On three occasions…” (l’enfant car indentation par rapport au parent) avec “Ronaldo performs better…” (le parent) et comme l’enfant est PRO alors on labelise 1, une autre paire serait “Across all his Champions League matches…” (en tant qu’enfant) avec “Ronaldo performs better in decisive matches… (le parent).

C’est l’enfant qui permet de labelisé selon son argument car c’est toujours l’enfant qui réagit au parent et pas l’inverse, il est favorable ou non à l’idée du parent, résultat on remonte le fichier jusqu’à arrivé à la racine tout en haut qui n’a pas de parent donc pas de création de paire supplémentaire.
Pour continuer la mémorisation **TF-IDF** transforme le texte en matrice afin de savoir quelles sont les parties du texte qui reviennent plus que les autres puis un modèle de régression logistique s'entraîne à désigner une probabilité de connectabilité par pair.

L’Accuracy est ensuite jugée sur des débats totalement inconnus pour voir à quel pourcentage l’algorithme arrive à avoir un lien ou non.

La **baseline** choisie est celle de la classe majoritaire (**dummy**), elle permet de simplement prendre l’accuracy des réponses les plus présentes. Elle sert de point de départ pour voir si les autres modèles sont intéressant en comparatif car elle ne regarde même pas le texte elle se contente de prédire à l’aveugle.

La **régression logistique** va se baser sur le **TF-IDF** (term frequency-inverse document frequency) qui affecte des poids selon la fréquence d’apparition d’un mot puis grâce à la fonction sigmoïde on passe la fréquence dans une formule qui devient une probabilité.

Le **MLP** prend également les vecteurs TF-IDF en entrée mais en les faisant passer par des couches de neurones (de **128** à **64**).

Pour éviter l’**overfitting** le max des **features** est plafonné à **5000** car au dessus l’efficacité de l'entraînement commence à converger et le rendement par rapport au temps passé est pas assez bénéfique.

Le **linear SVM** est le modèle ayant obtenu la meilleure accuracy ce qui est explicable par sa capacité à trouver les arguments les plus ambiguës, ceux qui se trouvent tout juste entre PRO et CON, étant donné qu’il y’a souvent des mots qui reviennent peu de fois et qu’il les ignores le score augmente. Il atteint environ 78% d’accuracy ce qui équivaut à environ 23% de plus que la baseline ce qui s’explique par les poids attribués par le SVM qui permettent de faire la part entre données utiles et inutiles dans toutes les dimensions ainsi l’espace vectoriel se combine très bien avec le TF-IDF qui permet d’obtenir des matrices immenses de données. Nos données sont linéaires car on n’a pas besoin dans ce cas précis de

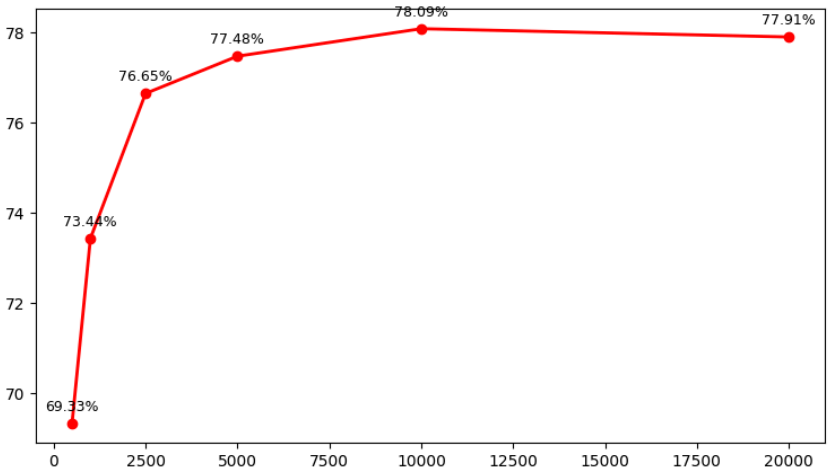
combinaison de mots, chaque mot est évalué de manière indépendante avec un poids lui étant attribué.

Voici le classement des modèles tester filtrer par ordre d'accuracy

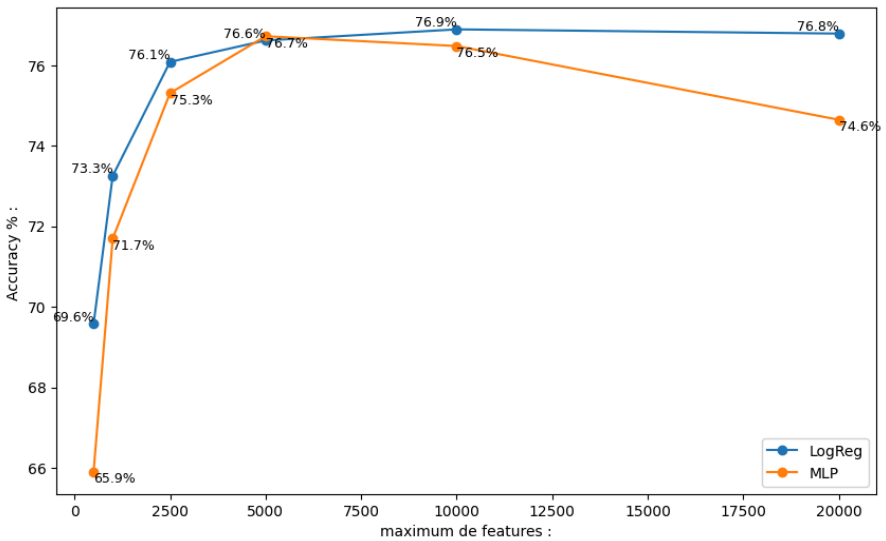
```
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
 CLASSEMENT MODÈLES
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
                Modèle  Accuracy
            Linear SVM  0.774808
Logistic Regression  0.766139
                   MLP  0.761608
              Baseline  0.552834
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

Voici un graphique représentant l'**overfitting** sur le **linear SVM**, entre 2500 et 10000 features le score n'augmente même pas de **2%** et de plus, de 10000 à 20000 il vient à baisser car le modèle va plus être déstabilisé par le bruit. On voit que la régression logistique **converge** car n'étant pas trop impactée par le bruit (poids ne bouge pas trop après beaucoup de features) tandis que le **MLP** (**réseaux de neurones** du **MLP** impacté par le bruit et la **complexité** qui augmente) dégringole dans le deuxième graphe.

Graphe représentant le pourcentage d'accuracy de linear SVM (axe y) selon le nombre de feature (axe x)



Graphe représentant le pourcentage d'accuracy (axe y) de MLP et Linear regression selon le nombre de feature (axe x)



Part 3: Visualising acceptability degree spaces for gradual semantics

This part examines how initial argument weights are transformed into acceptability scores. The aim is to visualise the resulting space, identify high and low density regions and assess whether the observations persist under resampling. All results use N = 100 000 weight vectors, seeds 2026-2030, ε = 10-8 .

Question 1: Weighted h-categorizer with a convergence parameter ε

We iterate the recursion HC□₊₁(a) = w(a) / (1 + Σ_{b∈Att(a)} HC□(b)), starting from HC₀ = w, updating all arguments simultaneously. We stop as soon as max_a |HC□₊₁(a) - HC□(a)| ≤ ε.

The attack relation is stored as a matrix M with M[t, s] = 1 if and only if s attacks t, so that Σ_{b∈Att(a)} x_b = (Mx)_a. A whole batch of weight vectors (one per row) is processed at once. The stopping test uses the maximum change over all rows, so every sample is converged to within ε. An iteration cap (10 000) raises an error instead of looping forever, and the inputs are validated (w ∈ [0,1], ε > 0, distinct arguments).

```
def h_categorizer(arguments, attacks, weights, epsilon=EPSILON):
    index = {a: i for i, a in enumerate(arguments)}
    if not arguments or len(index) != len(arguments):
        raise ValueError("Arguments must be non empty and distinct")
    M = np.zeros((len(arguments), len(arguments)))
    for source, target in attacks:
        M[index[target], index[source]] = 1
    w = np.asarray(weights, dtype=float)
    if w.ndim not in (1, 2) or w.shape[-1] != len(arguments) or w.size == 0:
        raise ValueError("Weights must match the number of arguments")
    if not np.isfinite(w).all() or np.any((w < 0) | (w > 1)):
        raise ValueError("Weights must be in [0, 1]")
    if not np.isfinite(epsilon) or epsilon <= 0:
        raise ValueError("epsilon must be positive and finite")
    x = w.copy()
    for iteration in range(1, 10_001):
        new_x = w / (1 + x @ M.T)
        if np.max(np.abs(new_x - x)) <= epsilon:
            return new_x, iteration
        x = new_x
    raise RuntimeError("The iteration did not converge")
```

Three cases with a closed form are asserted in the notebook:

- no attack: x = w;
- a → b with w = (0.2, 0.5): x = (0.2, 0.5/1.2) = (0.2, 0.4167)
- self-attack with w = 1: the fixed point solves x = 1/(1+x), so x = (√5 - 1)/2 ≈ 0.618.

All three pass. With ε = 10^{-8}, convergence takes 1-2 iterations without a cycle, 3 for the chain, and 20-27 when attacks are cyclic (Table 1). Cyclic frameworks are slowest because the effect of an update travels around the cycle.

Question 2: Acceptability degree spaces

(a, b) Frameworks and sampling

The program takes (A, R) as a list of arguments and attacks. It draws w_i uniformly in [0,1]^|A| (rng.random((N,|A|)), N = 100 000), then computes x_i = Σ_HC(w_i). The same seed is used for all frameworks so the inputs are comparable.

We study seven frameworks:

- with 2 arguments: no attacks, one attack a → b, mutual attack, self-attack on a;
- with 3 arguments: chain a → b → c, cycle, and complete graph (all attacks between distinct arguments).

(c) Convex hull

We build the hull with scipy.spatial.ConvexHull (Qhull) on the 100 000 points x_i: a polygon in 2D, a polyhedron in 3D.

Framework	Arguments	Iterations	Hull measure	Mean score
No attacks	2	1	0.99965	0.49834
One attack	2	2	0.74585	0.42195
Mutual attack	2	20	0.61590	0.37991
Self-attack	2	20	0.61775	0.42295
Chain	3	3	0.58965	0.40827
Cycle	3	20	0.47031	0.37157
Complete	3	27	0.30171	0.31874

Table 1: baseline run (seed 2026)

For 2D frameworks, the exact hull can be derived from its extreme points. The sampled hull agrees to within 0.5%, and it is always slightly smaller because it only sees sampled points.

- No attacks: $[0,1]^2$, area 1.
- One attack: vertices (0,0), (1,0), (1,½), (0,1), area 0.75.
- Mutual attack: vertices (0,0), (1,0), (φ,φ), (0,1) with φ = (√5 - 1)/2, area φ ≈ 0.618.
- Self-attack: [0,φ] × [0,1], area 0.618.

Attacks shrink the space: each attack lowers the target's score, and the more (and the more cyclic) the attacks, the smaller the hull. The complete graph has the smallest volume (0.30). Mutual attacks exclude joint high scores: both arguments can never be close to 1 together (at most (φ,φ)).

Caveat: the hull overapproximate. The hull is convex, but the attainable set is not. For a → b, the points (0,1) and (1,½) are attainable, but their midpoint (0.5, 0.75) would need w_b = 0.75 × 1.5 = 1.125 > 1. The true boundary is the curve x(b) = 1/(1 + x(a)), which lies below the hull's edge. This gap is visible in Figure 1 and does not disappear with more samples.

(d) Heat map of the density

2D. The heat map is a 2D histogram with 48 × 48 bins, normalised to a probability density. The same colour scale is used in all panels, cells without samples are grey, and the hull is drawn in black.

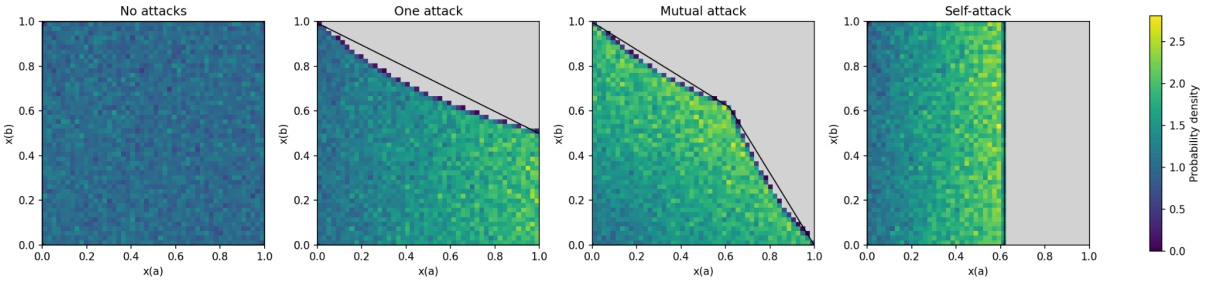


Figure 1: Density of (x(a), x(b)) for the 2 arguments frameworks, with a convex hull.

The patterns are explained analytically, which confirms the implementation.

- With no attacks, the density is uniform (= 1).
- With a → b, x(a) = w(a) and, given x(a), x(b) = w(b)/(1 + x(a)) is uniform on [0, 1/(1 + x(a))]. The joint density is therefore 1 + x(a) on {x(b) ≤ 1/(1 + x(a))}, rising from 1 to 2 towards x(a) = 1, as in the figure.
- With a self-attack, w(a) = x(a)(1 + x(a)) gives a marginal density 1 + 2x(a) on [0, φ] (from 1 to ≈ 2.24), independent of x(b), which is uniform.

We checked both numerically with 8 bins in x(a). For the self-attack the empirical values are 1.09 … 2.14 against 1.08 … 2.16 predicted. For the single attack they are 1.08 … 1.93 against 1.06 … 1.94 predicted. For mutual attack, the mass is compressed towards the upper-right boundary of the region.

3D. The 3D hull is drawn in full. Each face is coloured by a local density estimate, 300 / (N × (4/3)π r^3), where r is the distance from the face centre to its 300th nearest sample (cKDTree). This estimate is biased on the boundary, because half of the ball lies outside the set, and it smooths across empty gaps. The colours therefore show relative, not absolute, density.

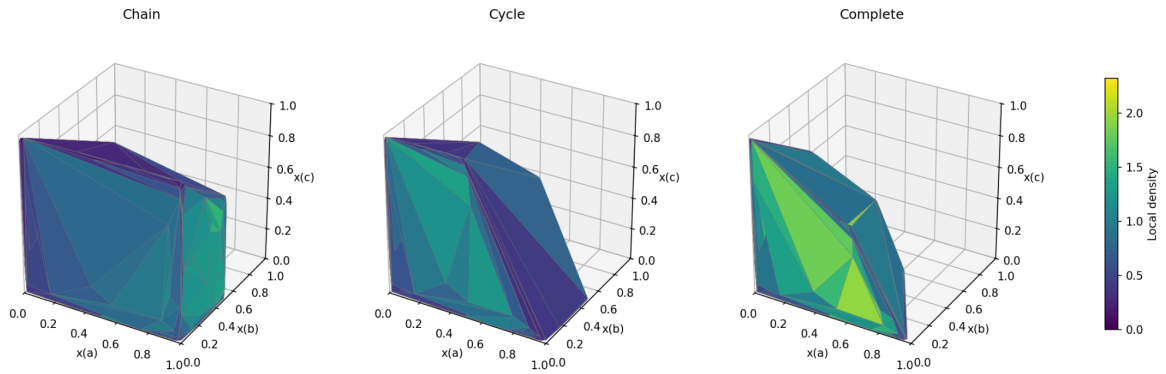


Figure 2: Convex hulls for the 3-argument frameworks, coloured by local density.

Is the heat map resilient to resampling? We repeated each experiment with four other seeds. On a fixed grid, we compare the bin proportions of each new sample to the baseline with the total variation TV(p, q) = ½ Σ_b |p_b - q_b| (0 = identical). We use 8, 16 and 32 bins per coordinate.

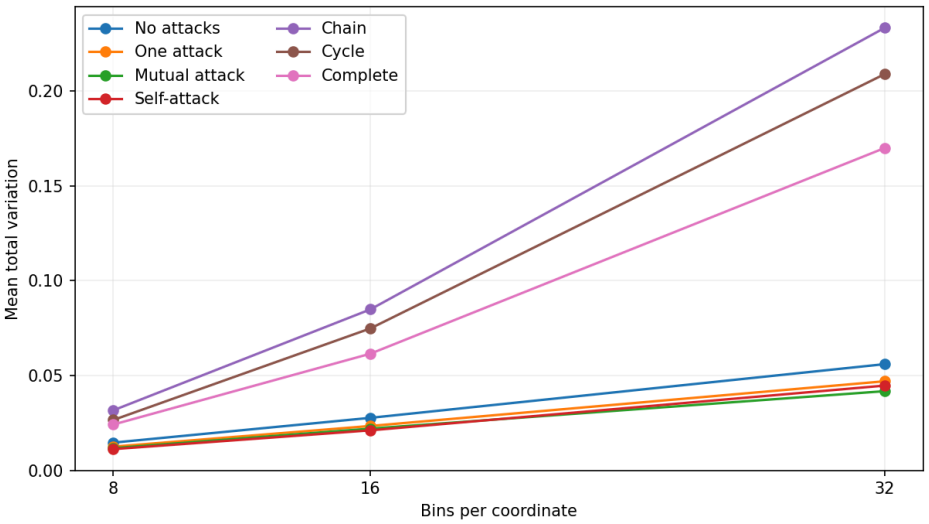


Figure 3: Mean TV between resampled and baseline histograms, as a function of the number of bins.

Raw reading. TV is small for 2D frameworks (0.011-0.056) and larger for 3D ones (up to 0.23 with 32 bins), and it increases with the number of bins. Strikingly, “no attacks” has a larger TV than the attacked 2D frameworks. This is not a sign of instability. TV between two finite samples is never zero, even for identical distributions, and it grows with the number of occupied bins K, since N points are spread over more cells.

Sanity check of the TV level. For a bin of probability p, the difference of two independent proportions has standard deviation $\sqrt{(2p(1-p)/N)}$. This gives $E[TV] \approx \sqrt{(K/(\pi N))}$ when the mass is spread evenly over K occupied bins. For “no attacks”, all K = 64, 256, 1024 bins are occupied, so the prediction is 0.014, 0.029, 0.057. We observe ≈ 0.015, 0.028, 0.056 (Figure 3).

The observed TV is therefore what pure sampling noise produces: resampling changes the histograms by sampling noise only, and the underlying density does not change from seed to seed. The same reasoning explains the other curves. Attacked 2D frameworks occupy fewer bins (part of the square is unattainable), hence a slightly lower TV than “no attacks”. 3D frameworks have far more bins, hence a much larger TV. This is why TV must not be compared across dimensions or bin counts.

Hull stability. Table 2 gives the range of the hull measure over the five seeds: at most 0.81% for every framework, and ≈ 0.01% for “no attacks” and “self-attack”. The hull depends only on a few extreme points and is therefore very stable.

Framework	mean	std	Hull range (%)
No attacks	0.02776	0.00124	0.01064
One attack	0.02346	0.00144	0.37148
Mutual attack	0.02207	0.00111	0.18223
Self-attack	0.02117	0.00156	0.01341
Chain	0.08490	0.00125	0.57136
Cycle	0.07487	0.00118	0.48030
Complete	0.06155	0.00167	0.80667

Table 2: re-sampling with the four other seeds, compared with the baseline (16 bins per coordinate)

Number of arguments. We vary the number of arguments from 2 to 3 (four 2 arguments and three 3 arguments frameworks). The 3 arguments frameworks are less resilient at the cell level for the reason above, but the TV across seeds is very regular (std ≈ 0.001-0.002 for all frameworks), so the behaviour is consistent. Larger frameworks were not tested. The hull can still be computed in higher dimension, but it cannot be displayed.

Conclusion.

The large-scale structure of the heat map (shape, gradient direction, location of dense and empty regions) is fully reproducible and matches closed-form densities where available. The pixel-level speckle is not: with 48 bins per axis, a 2D bin holds about 100 000 × ρ / 2304 ≈ 45ρ points, a relative fluctuation of $1/\sqrt{(45\rho)}$ ≈ 10-15%. Resilience decreases as the dimension grows. To keep the same noise level, the number of samples must grow with the number of occupied bins, so 3D heat maps should be read with coarser bins. The hull is stable, but it is a poor guide to the attainable space, because it is convex while the true space is not. The heat map reveals the gap, the hull alone does not.

Conclusion générale

Ce TP a permis de couvrir plusieurs aspects complémentaires de l’implémentation et de l’utilisation de la théorie de l’argumentation :

→ implémentation avec la partie 1 et les questions des choix de structures

→ prédiction classification d'arguments pour la partie 2

→ calcul d’acceptabilité et visualisation d’arguments avec la partie 3